

BookBinder User Guide ## From Markdown to Beautiful PDFs *The complete guide to installing, using, and extending BookBinder — the Markdown-to-PDF book generator with semantic markers, templates, and themes.*

*For everyone who believes documentation
should be as beautiful as the code it
describes.*

Contents

01	Installation
02	Usage
03	Standard Markdown Syntax
04	Extended Markdown Syntax
05	Name
06	Templates and Themes

Installation

Installation

BookBinder converts Markdown files into professionally formatted PDFs. This chapter covers every way to get it running on your system.

Prerequisites

BookBinder requires:

Component	Minimum Version	Purpose
Python	3.10+	Core runtime
pip	Latest	Package management
Chromium	(via Playwright)	PDF rendering engine
Java	11+ (<i>optional</i>)	PlantUML diagram rendering

NOTE

BookBinder uses Playwright's bundled Chromium for PDF generation — you do not need Chrome or Chromium installed system-wide.

Quick Start (Self-Bootstrap)

The fastest way to get running is BookBinder's built-in bootstrap command:

```
python make_a_book.py --self-bootstrap
```

This single command will:

1. Check for missing Python packages
2. Install `markdown` , `jinja2` , `playwright` , `pyyaml` , and `pygments`
3. Download and install Playwright's Chromium browser

After bootstrap completes, you can immediately build books.

 TIP

If you already have the dependencies but want to upgrade them all, use `--force-dependencies` instead.

Full Bootstrap Script

For a fresh system where Python itself may not be installed, use the standalone bootstrap script:

```
git clone https://github.com/yazici/BookBinder.git
cd BookBinder
chmod +x bootstrap.sh
./bootstrap.sh
```

Bootstrap Options

Flag	Effect
<code>--minimal</code>	Skip optional tools (Java/PlantUML)
<code>--upgrade</code>	Force-upgrade all Python dependencies
<code>--help</code>	Show usage information

What the Script Does

The bootstrap script performs these steps in order:

1. **Detects your OS and package manager** — supports apt, dnf, yum, pacman, zypper (Linux) and Homebrew (macOS)
2. **Installs Python 3.10+** — if not already present
3. **Ensures pip is available** — uses `ensurepip` or downloads `get-pip.py`

4. **Installs Python packages** — `markdown` , `jinja2` , `playwright` , `pyyaml` , `pygments`
5. **Installs Playwright Chromium** — including system dependencies on Linux
6. **Installs Java** (unless `--minimal`) — for PlantUML diagram rendering
7. **Verifies everything** — runs checks and reports status

! IMPORTANT

On Linux, Playwright's system dependency installation (``playwright install-deps``) requires `sudo`. The script will prompt for your password.

Manual Installation

If you prefer to install components yourself:

```
# 1. Install Python packages
pip install markdown jinja2 playwright pyyaml pygments

# 2. Install Playwright Chromium
python -m playwright install chromium

# 3. (Optional) Install Java for PlantUML
# Debian/Ubuntu:
sudo apt install default-jre-headless
# macOS:
brew install openjdk
```

Installation from Git

```
git clone https://github.com/yazici/BookBinder.git
cd BookBinder
python make_a_book.py --self-bootstrap
```

BookBinder is a standalone tool — no `setup.py` or `pip install -e .` required. Just clone and run `make_a_book.py` from the repository root.

Verifying Your Installation

Run the following to confirm everything is working:

```
# Check Python version
python3 --version

# Check that all imports work
python3 -c "import markdown, jinja2, playwright, yaml, pygments; print('All OK!)"

# Check Playwright Chromium
python3 -c "
from playwright.sync_api import sync_playwright
from pathlib import Path
with sync_playwright() as pw:
    p = Path(pw.chromium.executable_path)
    print(f'Chromium: {p}')
    assert p.is_file(), 'Chromium not found!'
print('Playwright OK')
"

# List available templates and themes
python3 make_a_book.py --list-templates
python3 make_a_book.py --list-themes
```

Platform Notes

Linux

On Debian/Ubuntu, Playwright requires several system libraries for Chromium. The bootstrap script handles this automatically via `playwright install-deps`. If you install manually, run:

```
sudo python3 -m playwright install-deps chromium
```

macOS

Homebrew is required for the bootstrap script. Install it from brew.sh if not present. Python 3.12 is recommended:

```
brew install python@3.12
```

Windows (WSL)

BookBinder runs under Windows Subsystem for Linux. Use the same Linux instructions inside your WSL distribution. Native Windows is not currently supported.

WARNING

Running BookBinder directly on Windows (outside WSL) is not supported due to Playwright Chromium path differences. Use WSL2 with Ubuntu for the best experience.

Updating BookBinder

To update to the latest version:

```
cd BookBinder
git pull
python make_a_book.py --force-dependencies
```

This pulls the latest code and upgrades all Python dependencies.

Usage

Usage

This chapter covers how to use BookBinder from the command line — from the simplest one-liner to advanced multi-file book projects.

Basic Usage

The simplest invocation converts a single Markdown file to PDF:

```
python make_a_book.py BOOK.md
```

This reads `BOOK.md`, processes all semantic markers, applies the default template and theme, and writes a PDF to the same directory. The output filename is derived from the book's title metadata.

Command-Line Options

```
python make_a_book.py [OPTIONS] SOURCE.md
```

Option	Description
<code>SOURCE.md</code>	The Markdown file to convert (required)
<code>-o, --output PATH</code>	Explicit output path (overrides auto-naming)
<code>--template NAME</code>	Layout template: <code>default</code> , <code>modern</code> , <code>technical</code>
<code>--theme NAME</code>	Color theme: <code>default</code> , <code>warm</code> , <code>cool</code> , <code>dark</code> , <code>minimal</code> , <code>academic</code>
<code>--paper SIZE</code>	Paper size: <code>A4</code> , <code>Letter</code> , <code>A3</code> , <code>Legal</code>
<code>--orientation DIR</code>	<code>portrait</code> or <code>landscape</code>
<code>--no-cover</code>	Skip the cover page
<code>--html-only</code>	Output HTML instead of PDF
<code>--self-bootstrap</code>	Install missing dependencies
<code>--force-dependencies</code>	Upgrade all dependencies
<code>--list-templates</code>	List available templates and exit
<code>--list-themes</code>	List available themes and exit
<code>--verbose</code>	Enable verbose error output

Examples

Generate a PDF with defaults

```
python make_a_book.py my_book/BOOK.md
```

Output: `my_book/MyBookTitle.pdf` (filename derived from `@book-meta` title)

Specify output path

```
python make_a_book.py BOOK.md -o /tmp/handbook.pdf
```

Choose a template and theme

```
python make_a_book.py BOOK.md --template modern --theme cool
```

Generate HTML for debugging

```
python make_a_book.py BOOK.md --html-only
```

This produces an HTML file you can open in any browser to inspect layout, styling, and content before committing to PDF.

TIP

Use `--html-only` during development to iterate quickly. HTML renders instantly while PDF generation takes a few seconds.

US Letter paper, landscape

```
python make_a_book.py BOOK.md --paper Letter --orientation landscape
```

Skip the cover page

```
python make_a_book.py BOOK.md --no-cover
```

Project Structure

A BookBinder project is simply a directory containing a `BOOK.md` file and optional chapter files:

```
MyBook/
├─ BOOK.md           ← Main entry point
├─ chapters/
│   ├─ 01_intro.md
│   ├─ 02_getting_started.md
│   ├─ 03_advanced.md
│   └─ ...
└─ figures/          ← Optional images/diagrams
    ├─ architecture.png
    └─ workflow.puml
```

NOTE

The directory structure is a convention, not a requirement. BookBinder resolves `@include` paths relative to the file containing the directive. You can organize files however you prefer.

The BOOK.md File

`BOOK.md` is the entry point. It typically contains:

1. **Metadata block** — title, author, theme selection
2. **Cover section** — front cover content
3. **Table of contents** — auto-generated
4. **Include directives** — pulling in chapter files
5. **Index** — auto-generated from index terms

Here's a minimal example:

```

<!-- @book-meta
title: "My Project Handbook"
subtitle: "A Complete Reference"
author: "Your Name"
version: "1.0"
date: "2026"
-->

<!-- @cover -->
# My Project Handbook
## A Complete Reference
<!-- @end-cover -->

<!-- @toc -->

<!-- @include: chapters/01_intro.md -->
<!-- @include: chapters/02_details.md -->

<!-- @index -->

```

Chapter Files

Each chapter file is a standalone Markdown file. It typically starts with a chapter marker:

```

<!-- @chapter: Introduction -->

# Introduction

Your chapter content here...

```

The `@chapter` marker creates a page break, registers the chapter in the table of contents, and sets the current chapter context for figure numbering and index terms.

The Build Pipeline

When you run `make_a_book.py`, the following pipeline executes:

1. **Markdown Processing** — Resolves `@include` directives recursively, extracts `@book-meta` YAML, processes all semantic markers into HTML-ready markup

2. **Template & Theme Loading** — Loads the CSS stylesheet and cover HTML templates based on metadata or CLI flags
3. **HTML Rendering** — Converts processed Markdown to HTML using Python-Markdown with extensions (tables, fenced code, syntax highlighting), injects TOC and index, wraps in a complete HTML document
4. **PDF Generation** — Launches headless Chromium via Playwright, loads the HTML, and prints to PDF with proper page sizing

Output Filename Logic

If you don't specify `-o`, BookBinder derives the output filename:

1. If `file_name` is set in `@book-meta`, that name is used
2. Otherwise, the `title` is converted to PascalCase with punctuation removed

Examples:

Title	Output Filename
"GDL User's Guide"	GDLUsersGuide.pdf
"My Project Handbook"	MyProjectHandbook.pdf
"BookBinder User Guide"	BookBinderUserGuide.pdf

The output file is always placed in the same directory as the source `BOOK.md`.

Working with Multiple Books

You can have multiple books in a single repository:

```
Books/
├── UserGuide/
│   ├── BOOK.md
│   └── chapters/
├── DevGuide/
│   ├── BOOK.md
│   └── chapters/
└── APIReference/
    ├── BOOK.md
    └── chapters/
```

Build each independently:

```
python make_a_book.py Books/UserGuide/BOOK.md
python make_a_book.py Books/DevGuide/BOOK.md
python make_a_book.py Books/APIReference/BOOK.md
```

TIP

Create a simple shell script or Makefile to build all books at once if your project has many.

Error Handling

If something goes wrong, BookBinder prints a concise error message. For full stack traces, add `--verbose` :

```
python make_a_book.py BOOK.md --verbose
```

Common errors:

Error	Cause	Fix
Missing dependency: ...	Python package not installed	Run <code>--self-bootstrap</code>
Source file not found	Wrong path to BOOK.md	Check the file path
Expected a Markdown file (.md)	Non-.md file passed	Rename or use correct file
Include depth exceeded 10 levels	Circular <code>@include</code>	Check for include loops
MISSING INCLUDE: path	Referenced file doesn't exist	Create the file or fix the path

Standard Markdown Syntax

Standard Markdown Syntax

BookBinder supports the full CommonMark Markdown specification plus several extensions. This chapter covers all standard formatting that works in any Markdown renderer — and in BookBinder's PDF output.

Headings

Use `#` characters for headings. BookBinder supports six levels:

```
# Heading 1
## Heading 2
### Heading 3
#### Heading 4
##### Heading 5
##### Heading 6
```

NOTE

In a BookBinder book, `# Heading 1` is typically used only for chapter titles. Use `###` through `####` for section structure within chapters.

Paragraphs and Line Breaks

Paragraphs are separated by blank lines. A single newline within text does not create a line break — it continues the same paragraph.

```
This is the first paragraph. It can span
multiple lines in the source file.
```

```
This is the second paragraph.
```

To force a line break within a paragraph, end the line with two spaces or use `<br>` :

```
Line one
Line two (forced break with trailing spaces)

Line one<br>
Line two (forced break with HTML)
```

Emphasis

```
*italic text* or _italic text_
**bold text** or __bold text__
***bold italic*** or ___bold italic___
~~strikethrough~~
```

Renders as: *italic*, **bold**, ***bold italic***, ~~~~strikethrough~~~~.

Lists

Unordered Lists

- Item one
- Item two
 - Nested item
 - Another nested item
- Item three

You can also use `*` or `+` as bullet markers.

Ordered Lists

1. First item
2. Second item
 1. Nested numbered item
 2. Another nested item
3. Third item

TIP

Markdown auto-numbers ordered lists. You can use `1.` for every item and the renderer will number them correctly. This makes reordering easier.

Task Lists

- ☒ Completed task
- ☐ Incomplete task
- ☐ Another pending task

Links

```
[Link text](https://example.com)
[Link with title](https://example.com "Hover title")
<https://example.com>
```

Reference-style links keep your text readable:

```
Read the [BookBinder docs][docs] for more information.

[docs]: https://github.com/yazici/BookBinder
```

Images

```
![Alt text](path/to/image.png)
![Alt text](path/to/image.png "Optional title")
```

Images are resolved relative to the Markdown file containing them. For more control over figures (captions, numbering, cross-references), use BookBinder's `@figure` marker described in the Extended Syntax chapter.

Blockquotes

```
> This is a blockquote.
> It can span multiple lines.
>
> > Nested blockquotes are supported.
```

Renders as:

This is a blockquote. It can span multiple lines.

Nested blockquotes are supported.

Code

Inline Code

```
Use the `make_a_book.py` script to build your PDF.
```

Fenced Code Blocks

Use triple backticks with an optional language identifier for syntax highlighting:

```
```python
def hello():
 print("Hello, BookBinder!")
```
```

BookBinder uses Pygments for syntax highlighting. Supported languages include:

| Language | Identifier |
|------------|--|
| Python | <code>python</code> |
| JavaScript | <code>javascript</code> or <code>js</code> |
| TypeScript | <code>typescript</code> or <code>ts</code> |
| Bash/Shell | <code>bash</code> or <code>shell</code> |
| C++ | <code>cpp</code> or <code>c++</code> |
| C | <code>c</code> |
| Java | <code>java</code> |
| Rust | <code>rust</code> |
| Go | <code>go</code> |
| YAML | <code>yaml</code> |
| JSON | <code>json</code> |
| HTML | <code>html</code> |
| CSS | <code>css</code> |
| SQL | <code>sql</code> |
| Markdown | <code>markdown</code> or <code>md</code> |

And many more — any language Pygments supports will work.

Indented Code Blocks

Four spaces of indentation also creates a code block (though fenced blocks are preferred):

```
def indented_code():  
    return "This is a code block"
```

Tables

Tables use pipes and hyphens:

```
Column 1	Column 2	Column 3
Cell 1	Cell 2	Cell 3
Cell 4	Cell 5	Cell 6
```

Column Alignment

Use colons in the separator row:

```
Left	Center	Right
L	C	R
L	C	R
```

| Left | Center | Right |
|------|--------|-------|
| L | C | R |
| L | C | R |

NOTE

Tables in BookBinder PDFs are styled by the active template. The `technical` template uses compact tables with alternating row colors.

Horizontal Rules

Three or more hyphens, asterisks, or underscores on a line:

```
---  
***  
---
```

All produce a horizontal rule. In BookBinder PDFs, these render as subtle dividers styled by the active theme.

HTML in Markdown

BookBinder passes through inline HTML. This is useful for advanced formatting:

```
<div style="text-align: center;">  
  <em>Centered italic text</em>  
</div>
```

WARNING

Inline HTML works but may not style consistently across templates. Prefer Markdown syntax and BookBinder's semantic markers when possible.

Escaping Special Characters

Use backslashes to escape Markdown syntax characters:

```
\*not italic\*
\# not a heading
\[not a link\]
\`not code\`
```

Characters that can be escaped: `\` ``` `*` `_` `{}` `[]` `()` `#` `+` `-` `.` `!` `|`

Footnotes

Here is a statement that needs a citation^[^1].

^[^1]: This is the footnote content.

Footnotes are collected and rendered at the bottom of the page (or section, depending on the template).

Definition Lists

Term 1
: Definition of term 1

Term 2
: Definition of term 2
: Alternative definition

Abbreviations

The HTML specification is maintained by the W3C.

*[HTML]: Hyper Text Markup Language

*[W3C]: World Wide Web Consortium

When rendered, hovering over "HTML" or "W3C" shows the full expansion.

Summary

BookBinder's standard Markdown support covers everything you need for technical writing:

Feature	Syntax
Headings	<code># H1</code> through <code>##### H6</code>
Bold	<code>**text**</code>
Italic	<code>*text*</code>
Code (inline)	<code>`code`</code>
Code (block)	<code>``` fences</code>
Links	<code>[text](url)</code>
Images	<code>![alt](path)</code>
Lists	<code>- item</code> or <code>1. item</code>
Tables	Pipe syntax
Blockquotes	<code>> text</code>
Horizontal rule	<code>---</code>

For BookBinder-specific features like includes, figures with captions, callouts, and more, see the next chapter on Extended Syntax.

Extended Markdown Syntax

Extended Markdown Syntax

BookBinder extends standard Markdown with **semantic markers** — special HTML comments that are invisible in normal Markdown renderers but control book structure, layout, and formatting when processed by BookBinder.

All markers use the format:

```
<!-- @marker-name: value -->
```

or for block markers:

```
<!-- @marker-start -->
content
<!-- @end-marker-start -->
```

Because they are HTML comments, your Markdown files remain valid and render normally in GitHub, VS Code preview, or any other Markdown viewer.

Book Metadata

The `@book-meta` block defines your book's metadata as YAML inside an HTML comment:

```
<!-- @book-meta
title: "My Book Title"
subtitle: "An Informative Subtitle"
author: "Author Name"
version: "2.0"
date: "2026"
theme: warm
template: modern
accent_color: "#b78a51"
cover_image: "covers/front.png"
publisher: "Publisher Name"
isbn: "978-0-000000-00-0"
language: "en"
file_name: "MyBookTitle"
-->
```

Available Metadata Fields

Field	Purpose	Default
<code>title</code>	Book title (used in cover, headers, filename)	"Untitled"
<code>subtitle</code>	Subtitle (cover and title page)	—
<code>author</code>	Author name(s)	—
<code>version</code>	Version string	—
<code>date</code>	Publication date	—
<code>theme</code>	Color theme name	"default"
<code>template</code>	Layout template name	"default"
<code>accent_color</code>	Override theme accent color (hex)	—
<code>cover_image</code>	Path to cover image	—
<code>publisher</code>	Publisher name (back cover)	—
<code>isbn</code>	ISBN (back cover)	—
<code>language</code>	HTML lang attribute	"en"
<code>file_name</code>	Explicit output filename (without extension)	—

TIP

You can also set the template outside the metadata block with a standalone `` directive anywhere in the file.

File Includes

The `@include` directive inserts the contents of another Markdown file:

```
<!-- @include: chapters/01_introduction.md -->
<!-- @include: chapters/02_getting_started.md -->
<!-- @include: appendix/glossary.md -->
```

Path Resolution

Paths are resolved **relative to the file containing the directive**. This means included files can themselves include other files with paths relative to their own location.

Recursive Includes

Includes are resolved recursively up to 10 levels deep:

```
<!-- In BOOK.md -->
<!-- @include: parts/part1.md -->

<!-- In parts/part1.md -->
<!-- @include: chapters/ch1.md -->
<!-- @include: chapters/ch2.md -->

<!-- In parts/chapters/ch1.md -->
<!-- @include: sections/intro.md -->
```

WARNING

Circular includes (A includes B which includes A) will trigger an error after 10 levels of recursion.

Missing Includes

If an included file doesn't exist, BookBinder inserts a visible placeholder:

```
<!-- MISSING INCLUDE: path/to/file.md -->
```

This renders in the output so you can spot missing files during development.

Cover Pages

Front Cover

```
<!-- @cover -->

# Book Title

## Subtitle

*Description or tagline*

<!-- @figure: covers/hero.png | Cover illustration -->

<!-- @end-cover -->
```

The content between `@cover` and `@end-cover` is wrapped in a full-page cover layout. If no inline cover is provided, BookBinder uses the template's `cover.html` Jinja2 template with metadata variables.

Back Cover

```
<!-- @back-cover -->

## About This Book

Brief description for the back cover.

<!-- @end-back-cover -->
```

Chapters and Structure

Chapter Breaks

```
<!-- @chapter: Getting Started -->
```

This creates: - A page break before the chapter - A chapter header with the title - A TOC entry - Sets the current chapter context (for figure numbering and index terms)

Page Breaks

```
<!-- @page-break -->
```

Forces a page break at that point in the output. Useful between major sections that don't warrant a full chapter marker.

Dedication Page

```
<!-- @dedication: To my family, who tolerated the sound of a mechanical keyboard at 2 AM. -->
```

Renders a centered dedication page with elegant typography.

Table of Contents

```
<!-- @toc -->
```

Generates a clickable table of contents from all `@chapter` markers. Place it after the cover and before the first chapter include.

The TOC is automatically populated — you don't need to maintain it manually.

Figures and Cross-References

Basic Figure

```
<!-- @figure: diagrams/architecture.png | System architecture overview -->
```

This renders a numbered figure with caption: "Figure 1. System architecture overview"

Figure with Explicit ID

```
<!-- @figure: diagrams/flow.png | Data flow diagram | data-flow -->
```

The third parameter sets an explicit ID for cross-referencing.

Figure Path Resolution

Figures are resolved in this order:

1. Absolute path (starts with `/`)
2. Relative to the chapter file's directory
3. Inside the chapter's `figures/` subfolder (bare filename shorthand)
4. Relative to the book root (`BOOK.md` directory)

Cross-Referencing Figures

```
As shown in <!-- @fig-ref: data-flow -->, the data flows through three stages.
```

Renders as: "As shown in Figure 2, the data flows through three stages." — with a clickable link to the figure.

PlantUML Diagrams

BookBinder can render `.puml` files directly:

```
<!-- @figure: diagrams/sequence.puml | Authentication sequence -->
```

If Java and PlantUML are available, the `.puml` file is rendered to SVG automatically. If a pre-rendered `.svg` file exists alongside the `.puml`, it's used without re-rendering.

NOTE

PlantUML rendering requires Java. If Java is not installed, BookBinder looks for a pre-rendered `.svg` with the same name. Include pre-rendered SVGs in version control for environments without Java.

Callouts

BookBinder provides four callout types for highlighting important information:

Tip

```
<!-- @tip: Use `--html-only` during development for faster iteration. -->
```



TIP

This is how a tip callout looks in the rendered output.

Note

```
<!-- @note: This feature requires BookBinder 1.0 or later. -->
```



NOTE

This is how a note callout looks in the rendered output.

Warning

```
<!-- @warning: This operation cannot be undone. Back up your data first. -->
```



WARNING

This is how a warning callout looks in the rendered output.

Important

```
<!-- @important: You must run bootstrap before first use. -->
```

! IMPORTANT

This is how an important callout looks in the rendered output.

Sidebar

```
<!-- @sidebar: Historical context – BookBinder was originally designed for game documentation. -->
```

Sidebars are styled as supplementary information panels, visually distinct from the main text flow.

Index

Marking Index Terms

Sprinkle index term markers throughout your text:

```
<!-- @index-term: authentication -->
<!-- @index-term: OAuth 2.0 -->
```

These are invisible in the output but register the term at that location.

Generating the Index

```
<!-- @index -->
```

Place this at the end of your book. It generates an alphabetical back-of-book index with page/section references for every marked term.

Layout Control

Two-Column Layout

```
<!-- @layout: two-column -->
```

Content here flows in two columns. Useful for reference tables, glossaries, or dense information.

```
<!-- @layout: single -->
```

Switch back to single-column with `

`.

Foreword

```
<!-- @foreword -->
```

This book represents three years of development experience...

```
*— Foreword Author*
```

```
<!-- @end-foreword -->
```

The foreword section receives special styling (typically italic, indented) appropriate for introductory material written by someone other than the main author.

Color Palette

Display a visual color palette (useful for design documentation):

```
<!-- @color-palette: #2c5a8a, #4a9eff, #f0f4f8, #1a3a5c, #ffffff -->
```

Renders as a row of color swatches with hex values.

Template Selection (Inline)

You can select a template outside the metadata block:

```
<!-- @template: technical -->
```

This overrides the template specified in `@book-meta` (but CLI `--template` takes highest priority).

Marker Reference

Complete list of all semantic markers:

Marker	Purpose
<code><!-- @book-meta ... --></code>	Book metadata (YAML)
<code><!-- @template: name --></code>	Select template
<code><div class="cover-page"><div class="cover-content"></code>	Start front cover
<code></div></div></code>	End front cover
<code><div class="back-cover page-break"><div class="back-cover-content"></code>	Start back cover
<code></div></div></code>	End back cover
<code><div class="foreword"></code>	Start foreword
<code></div></code>	End foreword

Contents

01	Installation	
02	Usage	
03	Standard Markdown Syntax	
04	Extended Markdown Syntax	
05	Name	
06	Templates and Themes	

Index

@

@chapter
@figure
@include

A

academic theme
admonitions

B

back-of-book index
book-meta
bootstrap

C

callouts
chapters
CLI
color palette
command line
cool theme
cover
cover.html
CSS custom properties
custom template
custom theme

D

dark theme
default template
default theme
diagrams

E

extended syntax

F

figures
foreword

I

includes
index
installation

L

layout

M

markdown
metadata
minimal theme
modern template

P

page orientation
paper size
pipeline
PlantUML

S

semantic markers
style.css
syntax
syntax highlighting

T

table of contents
technical template
template structure
templates
term
themes
TOC
two-column

U

usage

W

warm theme

Y

YAML theme

Name

| Chapter break | |

| Page break | |

caption

path \

Figure 1. caption

| Figure | |

**caption **

path \

*Figure 2. caption *

| Figure with ID | | **Figure 2** | Figure reference | |

 **TIP**

text

| Tip callout | |

 **NOTE**

text

| Note callout | |

 WARNING

text

| Warning callout | |

 IMPORTANT

text

| Important callout | |



text

| Sidebar callout | | | Index term | |

text

| Dedication page | |

` | Two-column layout | | `

| Single-column layout | |



colors

` | Color swatches |

Priority and Override Order

When the same setting is specified in multiple places, this priority applies (highest first):

1. **CLI flags** (`--template` , `--theme`)
2. **Inline directives** (`<!-- @template: ... -->`)
3. **Metadata block** (`@book-meta` YAML)
4. **Defaults** (template: "default", theme: "default")

Templates and Themes

Templates and Themes

BookBinder separates visual presentation into two layers: **templates** control layout and structure, while **themes** control colors and typography. This separation lets you mix and match — use the `technical` template with the `warm` theme, or the `modern` template with the `dark` theme.

Built-in Templates

BookBinder ships with three templates:

Template	Description	Best For
<code>default</code>	Classic book layout with serif typography	General-purpose books, manuals
<code>modern</code>	Clean sans-serif design with generous whitespace	User guides, marketing docs
<code>technical</code>	Compact layout with monospace accents, alternating table rows	API docs, developer guides

List available templates:

```
python make_a_book.py --list-templates
```

Selecting a Template

Three ways to select a template (in priority order):

```
# 1. CLI flag (highest priority)
python make_a_book.py BOOK.md --template modern

# 2. Inline directive in Markdown
<!-- @template: modern -->

# 3. In @book-meta YAML
<!-- @book-meta
template: modern
-->
```

Built-in Themes

BookBinder ships with six color themes:

Theme	Accent Color	Description
default	Blue (#2c5a8a)	Professional blue on white
warm	Golden (#b78a51)	Warm amber tones, inviting feel
cool	Teal (#2a7a8a)	Cool blue-green, modern
dark	Light blue (#4a9eff)	Dark background, light text
minimal	Gray (#555555)	Monochrome, distraction-free
academic	Deep red (#8b2252)	Traditional academic styling

List available themes:

```
python make_a_book.py --list-themes
```

Selecting a Theme

```
# CLI flag
python make_a_book.py BOOK.md --theme warm

# In @book-meta YAML
<!-- @book-meta
theme: warm
-->
```

Accent Color Override

Override just the accent color without creating a full custom theme:

```
<!-- @book-meta
theme: warm
accent_color: "#e67e22"
-->
```

This uses the `warm` theme's fonts and background but replaces the accent color.

Theme Anatomy

A theme is a YAML file defining CSS custom properties. Here's the complete structure:

```
# themes/my_theme.yaml
name: my_theme
description: A custom theme for my project

colors:
  accent: "#2c5a8a"      # Primary accent (headings, links, borders)
  accent-light: "#f0f4f8" # Light accent (backgrounds, callout fills)
  accent-dark: "#1a3a5c"  # Dark accent (hover states, emphasis)
  text: "#1a1a1a"         # Body text color
  bg: "#ffffff"           # Page background
  code-bg: "#f7f8fa"     # Code block background

fonts:
  heading: "'Georgia', serif"
  body: "'Charter', 'Georgia', serif"
  mono: "'JetBrains Mono', monospace"
```

These values become CSS custom properties:

```
:root {
  --accent: #2c5a8a;
  --accent-light: #f0f4f8;
  --accent-dark: #1a3a5c;
  --text: #1a1a1a;
  --bg: #ffffff;
  --code-bg: #f7f8fa;
  --font-heading: 'Georgia', serif;
  --font-body: 'Charter', 'Georgia', serif;
  --font-mono: 'JetBrains Mono', monospace;
}
```

Template Anatomy

A template is a directory containing three files:

```
templates/my_template/
├─ style.css      ← Main stylesheet
├─ cover.html     ← Front cover (Jinja2 template)
└─ back_cover.html ← Back cover (Jinja2 template)
```

style.css

The stylesheet uses CSS custom properties from the active theme. This means a single template works with any theme:

```
/* Use theme variables */
h1, h2, h3 {
  color: var(--accent);
  font-family: var(--font-heading);
}

body {
  color: var(--text);
  background: var(--bg);
  font-family: var(--font-body);
}

code, pre {
  font-family: var(--font-mono);
  background: var(--code-bg);
}

.callout {
  border-left: 4px solid var(--accent);
  background: var(--accent-light);
}
```

cover.html

A Jinja2 template for the front cover. Available variables:

Variable	Source
<code>{{ title }}</code>	<code>@book-meta title</code>
<code>{{ subtitle }}</code>	<code>@book-meta subtitle</code>
<code>{{ author }}</code>	<code>@book-meta author</code>
<code>{{ version }}</code>	<code>@book-meta version</code>
<code>{{ date }}</code>	<code>@book-meta date</code>
<code>{{ cover_image }}</code>	<code>@book-meta cover_image</code>
<code>{{ publisher }}</code>	<code>@book-meta publisher</code>
<code>{{ isbn }}</code>	<code>@book-meta isbn</code>

Example cover template:

```
<div class="cover-page">
  <div class="cover-content">
    {% if cover_image %}
    
    {% endif %}
    <h1 class="cover-title">{{ title }}</h1>
    {% if subtitle %}
    <h2 class="cover-subtitle">{{ subtitle }}</h2>
    {% endif %}
    {% if author %}
    <p class="cover-author">{{ author }}</p>
    {% endif %}
    {% if version %}
    <p class="cover-version">Version {{ version }}</p>
    {% endif %}
  </div>
</div>
```

back_cover.html

Same Jinja2 variables as the front cover. Typically shows publisher info, ISBN, and a brief description.

Creating a Custom Theme

1. Create a YAML file in `book_binder/templates/themes/` :

```
cp book_binder/templates/themes/default.yaml book_binder/templates/themes/my_project.yaml
```

2. Edit the colors and fonts:

```
name: my_project
description: Custom theme for My Project documentation

colors:
  accent: "#e74c3c"
  accent-light: "#fdf2f2"
  accent-dark: "#922b21"
  text: "#2c3e50"
  bg: "#ffffff"
  code-bg: "#f8f9fa"

fonts:
  heading: "'Inter', 'Helvetica Neue', sans-serif"
  body: "'Source Sans Pro', 'Helvetica', sans-serif"
  mono: "'Fira Code', 'Consolas', monospace"
```

3. Use it:

```
python make_a_book.py BOOK.md --theme my_project
```

TIP

Start by copying an existing theme that's close to what you want, then adjust colors. The `warm` theme is a good starting point for most custom themes.

Creating a Custom Template

1. Create a directory in `book_binder/templates/` :

```
mkdir book_binder/templates/my_layout
```

2. Create the three required files:

```
# Start from an existing template
cp book_binder/templates/modern/style.css book_binder/templates/my_layout/
cp book_binder/templates/modern/cover.html book_binder/templates/my_layout/
cp book_binder/templates/modern/back_cover.html book_binder/templates/my_layout/
```

3. Customize `style.css` for your layout needs. Key CSS classes to style:

CSS Class	Element
<code>.cover-page</code>	Front cover container
<code>.back-cover</code>	Back cover container
<code>.chapter-break</code>	Chapter title page
<code>.chapter-header</code>	Chapter heading
<code>.figure</code>	Figure container
<code>.figure-caption</code>	Figure caption text
<code>.callout</code>	Callout box
<code>.callout-tip</code>	Tip-specific styling
<code>.callout-note</code>	Note-specific styling
<code>.callout-warning</code>	Warning-specific styling
<code>.callout-important</code>	Important-specific styling
<code>.toc</code>	Table of contents
<code>.index-section</code>	Back-of-book index
<code>.two-column</code>	Two-column layout region
<code>.dedication-page</code>	Dedication page
<code>.foreword</code>	Foreword section
<code>.highlight</code>	Syntax-highlighted code
<code>.page-break</code>	Page break element
<code>.no-break</code>	Prevent page break inside

4. Use your template:

```
python make_a_book.py BOOK.md --template my_layout
```

Page Size and Orientation

Templates automatically adapt to the paper size and orientation specified via CLI:

```
python make_a_book.py BOOK.md --paper A4 --orientation portrait
python make_a_book.py BOOK.md --paper Letter --orientation landscape
```

Supported paper sizes:

Size	Dimensions
A4	210 × 297 mm
Letter	8.5 × 11 in
A3	297 × 420 mm
Legal	8.5 × 14 in

The paper size and orientation are available as CSS custom properties (`--paper-size` and `--orientation`) for templates that need to adjust layout based on page dimensions.

Design Guidelines

When creating custom templates and themes, keep these principles in mind:

For Themes

- **Contrast:** Ensure sufficient contrast between text and background (WCAG AA minimum)
- **Accent usage:** The accent color should be used sparingly — headings, links, borders, callout accents
- **Code readability:** `code-bg` should provide enough contrast for syntax-highlighted code
- **Print-friendliness:** If the book will be printed, avoid very dark backgrounds

For Templates

- **Use CSS custom properties:** Always reference `var(--accent)` , `var(--font-body)` , etc. instead of hardcoding colors. This ensures your template works with any theme.
- **Page breaks:** Use `page-break-before: always` on `.chapter-break` and `.page-break` elements
- **Avoid orphans/widows:** Use `orphans: 3; widows: 3;` on body text
- **Figure handling:** Figures should have `page-break-inside: avoid` to prevent splitting across pages
- **Responsive to paper size:** Test your template with both A4 and Letter sizes

NOTE

BookBinder's PDF engine (Playwright/Chromium) supports CSS Paged Media properties like `@page` , `page-break-before` , `page-break-after` , and `page-break-inside` . Use these for precise print layout control.

Example: Complete Custom Setup

Here's a complete example of a book using a custom theme with accent override:

```

<!-- @book-meta
title: "Project Phoenix Documentation"
subtitle: "Internal Developer Reference"
author: "Phoenix Team"
version: "3.1"
date: "2026"
template: technical
theme: cool
accent_color: "#e67e22"
file_name: "PhoenixDocs"
-->

<!-- @cover -->
# Project Phoenix
## Internal Developer Reference
*Version 3.1 – 2026*
<!-- @end-cover -->

<!-- @toc -->

<!-- @include: chapters/01_overview.md -->
<!-- @include: chapters/02_api.md -->
<!-- @include: chapters/03_deployment.md -->

<!-- @back-cover -->
## Project Phoenix
Internal documentation. Not for external distribution.
<!-- @end-back-cover -->

<!-- @index -->

```

Build with:

```
```bash python make_a_book.py BOOK.md
```

**Output: PhoenixDocs.pdf (technical template, cool theme, orange accent)**

# Index

---

## @

@chapter  
@figure  
@include

## A

academic theme  
admonitions

## B

back-of-book index  
book-meta  
bootstrap

## C

callouts  
chapters  
CLI  
color palette  
command line  
cool theme  
cover  
cover.html  
CSS custom properties  
custom template  
custom theme

## D

dark theme  
default template  
default theme  
diagrams

## E

extended syntax

## F

figures  
foreword

## I

includes  
index  
installation

## L

layout

## M

markdown  
metadata  
minimal theme  
modern template

## P

page orientation  
paper size  
pipeline  
PlantUML

## S

semantic markers  
style.css  
syntax  
syntax highlighting

## T

table of contents  
technical template  
template structure  
templates  
term  
themes  
TOC  
two-column

## U

usage

## W

warm theme

## Y

YAML theme

# BookBinder User Guide

BookBinder Contributors

1.0 — 2026

